University of Applied Sciences

HOCHSCHULE EMDEN·LEER

Checklist

Prof. Dr. E. Wings

Agenda

1. Prerequisite: Business & Data Understanding
2. I. Import Data & Libraries
3. II. Exploratory Data Analysis
4. III. Visualize & Understand
5. IV. Assess Missing Values
6. V. Train/Test Split
7. VI. Prepare for ML
8. VII. Feature Engineering
9. VIII. Transform Data
10. IX. Pick your Models
11. X. Model Selection
12. XI. Model Tuning
13. XII. Pick the Best Model
14. Concluding Thoughts

Prerequisite: Business & Data Understanding

Prerequisite: Business & Data Understanding

Data preparing is not the favorite job, but necessary.

1. The business part

Example: Assume your manager hands you an Airbnb dataset. It has 106 features, and she tells you that we want to better understand rental listings.

OK. First, does she want me to cluster them?
Does she want me to make a dashboard?
What questions does she want me to answer?
What's the goal?

$\Rightarrow$ Figure out what your objective is, and try to gameplan the different ways you can get to your objective.

1. The business part

Example: Assume your manager hands you an Airbnb dataset. It has 106 features, and she tells you that we want to better understand rental listings.

OK. First, does she want me to cluster them?
Does she want me to make a dashboard?
What questions does she want me to answer?
What's the goal?

$\Rightarrow$ Figure out what your objective is, and try to gameplan the different ways you can get to your objective.

1. The business part

Example: Assume your manager hands you an Airbnb dataset. It has 106 features, and she tells you that we want to better understand rental listings.

OK. First, does she want me to cluster them?
Does she want me to make a dashboard?
What questions does she want me to answer?
What's the goal?

$\Rightarrow$ Figure out what your objective is, and try to gameplan the different ways you can get to your objective.

2. The data part

Is there a data dictionary or schema you can print or keep handy? Take advantage of that.

Take 1-2 hours to understand what each of those 106 features are. Write down features you plan on removing, features whose names don't make intuitive sense, and features that look like they might need a lot of cleaning.

Is this event-level, where one listing can have multiple rows - each with a timestamp and an event? Or is this unit-level, where each row uniquely identifies one rentable unit?

2. The data part

Get familiar with what you're working with before you even start coding.

Understanding and answering questions like these will help you start moving in the right direction. It's like looking at a map before stepping into the maze.

I. Import Data & Libraries

Import Data & Libraries

It's good practice to keep all your imports at the top of your analysis. Whenever you need a library or function you haven't imported, go back to the top and import it there.

Import your dataset(s) now, and if you have to join them, try to wait until you've finished Exploratory Data Analysis (EDA) before you do so.

II. Exploratory Data Analysis

Exploratory Data Analysis

This section is broken up into 5 subsections because of how long it is.
You know how people say Data Scientists spend up to 80%-90% of their time cleaning data?
Yeah, that's mainly because of this part.

Make sure you take your time. Be thorough here. Once you finish this part, you're well beyond halfway to a working ML algorithm.

Check for Duplicates

Take a second to drop pure duplicate rows or investigate conflicting IDs from your dataset(s). It's easy to forget, but this is a step you should always take.

Separate your Data Types

Take an inventory of your data types. Generally, there are 6 data types that you might encounter, in decreasing order of frequency.

- Numerical (Can be Discrete or Continuous)
- Categorical (Ordinal, Nominal, and Binary)
- Date/Time
- Text
- Image
- Sound

Separate your Data Types

Start by separating your data into the numerical data types and everything else. This can be done with the following:

```
# Categorical data
cat_df = df.select_dtypes(['object', 'bool'])
# Numerical data
num_df = df.select_dtypes(['int', 'float'])
```

Skim through your numerical data first. Most often, the only problem here will be missing values (which we will deal with later).

Separate your Data Types

Now, look through your categorical data. There will probably be a lot of problems that you need to take care of. Some of them being ...

- Numerical data, Binary data, and Date/Time data are typed as strings
- Multiple values are packed into one feature
- Features might be incorrectly named
- Features that you thought sounded useful appear pointless

Separate your Data Types

Don't start cleaning as soon as you see a problem. Let's not get ahead of ourselves.

Make a to-do list of the problems you can see now. You might not catch them all, so if another cleaning problem comes up later, just add it to this list.

Separate your Data Types

Here some example:

1. Numerical data stored as Catgegorical (strings)
   → Convert some of these to numerical columns (i.e. `price` and `host_response_rate`)
2. Binary data needs to be binarized into 1's and 0's.
   → We can binarize the Binary/Boolean columns (such as `require_licence`)
3. Ordinal (should generally be encoded to retain their information (e.g. $\{1, 2, 3\}$ to encode $\{\text{low}, \text{med}, \text{high}\}$)
   → The only ordinal-looking column I see is `host_response_time`, but let's treat it as nominal for simplicity
4. Nominal data to be unpacked, then later one hot encoded
   - → `host_verification` and `amentities` have multiple items that need to be extrapolated into their own columns
   - → All other categorical columns, like `neighborhood`, `cancellation_policy`, `property_type` should be one hot encoded.
5. Date/Time features need to be engineered
   → Using these dates we can engineer features from the dates columns. We'll do this later.

Initial Data Cleaning

If you added „fill missing values“ to your cleaning to-do list, remove it for now. Don't do it yet. I'll explain why in a later step.

Once you have a comprehensive cleaning to-do list, finish them one at a time. Here's an example of how I unpacked a feature with multiple values.

Out[25]:

	host_verifications
0	['email', 'phone', 'reviews', 'kba']
1	['email', 'phone', 'reviews', 'kba']

Initial Data Cleaning

```
def striplist(l):
"'
To be used with the apply method on a packed feature
"'
return([x.strip() for x in l])

# This turns the string of the lists into actual Python lists
cat_df['host_verifications'] = cat_df['host_verifications'].str.replace('[', '') \
.str.replace(']', '') \
.str.replace("'","") \
.str.lower() \
.str.split(',') \
.apply(striplist)

# Using sklearn's MultiLabelBinarizer to one hot encode the lists
mlb = MultiLabelBinarizer()

# This creates a one-hot encoded matrix
host_verif_matrix = mlb.fit_transform(cat_df['host_verifications'])

# Converting to a DataFrame so you can see the output

host_verif_df = pd.DataFrame(host_verif_matrix, columns = mlb.classes_)
```

III. Visualize & Understand

Visualize & Understand

Note that the things you find in this section can and will uncover things that you will want to add to the cleaning to-do list.

If you identify any outliers or transformations you want to apply to your data, you can apply it now, but it's better practice to do this after you've split your data into training and testing.

If you want, check out how your data is distributed with histograms, distribution plots, or bar plots. Maybe explore relationships between features.

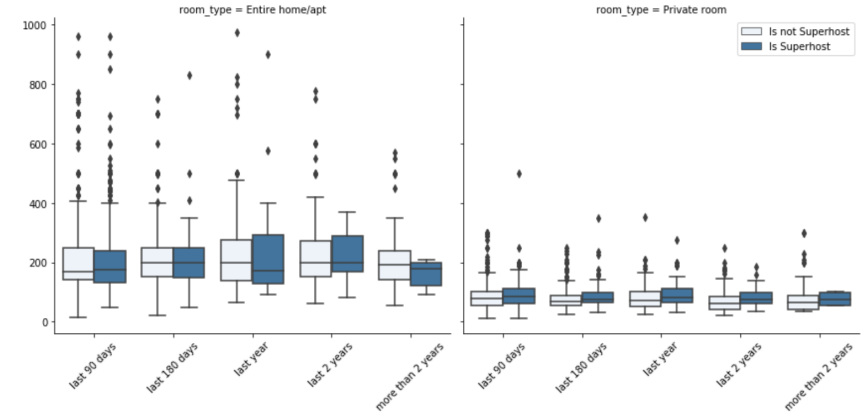
Visualize & Understand

Use this section to see if you can answer the original business questions that was asked of you. You can perform any Machine Learning yet, but not all problems require Machine Learning in the first place.

Visualize & Understand

Take advantage of data viz libraries like Matplotlib or Seaborn. Maybe explore interactive visualizations using Bokeh or Plotly. The world is your oyster.

Here's an example of how I tried to understand if there is some relationship between recency of reviews and the pricing of an Airbnb listing.



IV. Assess Missing Values

Assess Missing Values

Unless you're using a fancy algorithm like XGboost, you have to address missing values so your data is ML-friendly.

Put simply, you have two options: remove or impute.

In this section, remove missing values if you think it's appropriate, then strategize for how you want to fill them, but don't impute them yet.

Generally, it's good practice to treat the imputation of missing values as another hyperparameter that you want to tune. So, you want to run your imputation strategy after you train/test split your data. If you want, you can also treat your removal of missing values the same way (i.e. remove missing values after train/test split).

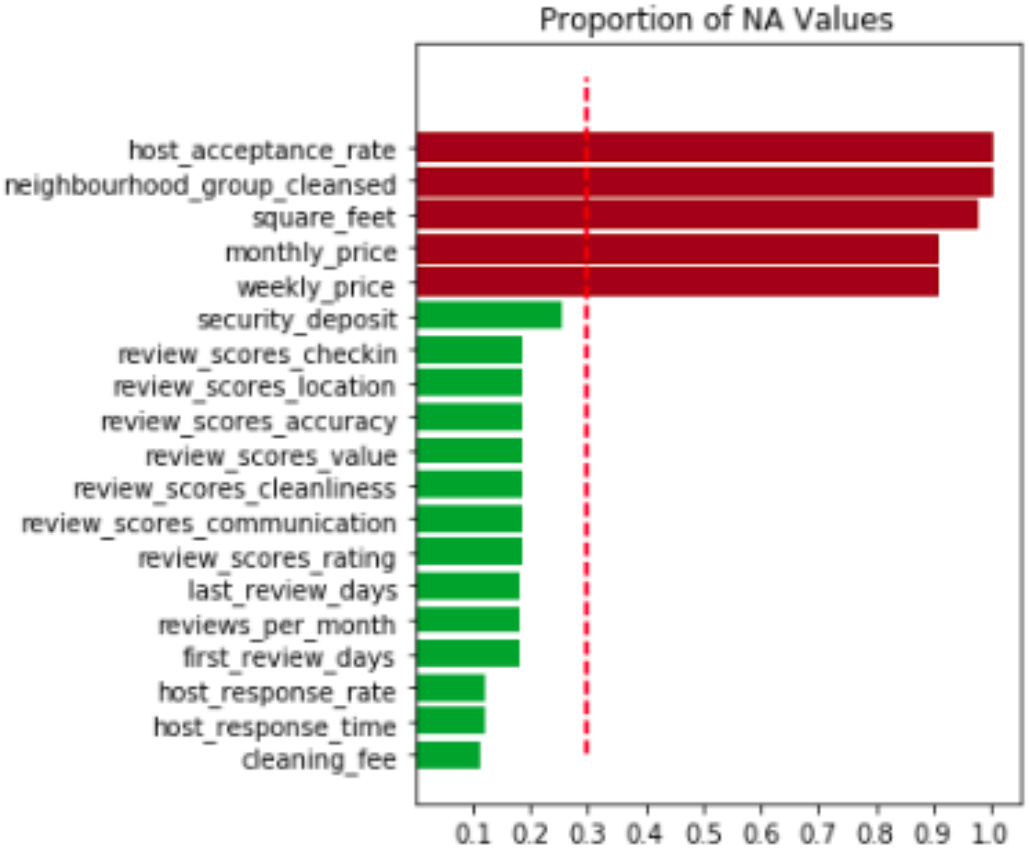
Assess Missing Values

This way, when you test your entire machine learning workflow on your validation set, you can also see how changing your imputation or removal strategy affects your workflow's performance.

E. Assess Missing Values - 1. Remove them

- Are there many missing values in particular features?
  Pick a threshold (say - 70%) and remove the features if you feel they're not valuable.
- Are there many missing values in particular rows?
  First, investigate why they may be missing in case you can think of a reasonable explanation for why rows have missing values. If you feel they're not valuable, drop them.
- Which rows are missing the target feature that you want to predict?
  Supervised learning methods require a label, so drop rows that don't contain your target variable.

Assess Missing Values - 1. Remove them



Features in red should probably be removed.

Assess Missing Values - 2. Impute them (strategize now, impute later)

- Infer their values from other features based on your knowledge
- Numerical? Filling with mean or median is straightforward.
- Categorical? Fill with most frequent.
- Want a challenge? Look into using Machine Learning to predict your missing values. Scikit-learn's `IterativeImputer` is interesting.

Assess Missing Values - 2. Impute them (strategize now, impute later)

For each strategy you use, make note of which features you want each strategy applied to. This makes imputation easier after you split your data.

Awesome, now we're done with the EDA portion of the project.

V. Train/Test Split

III. Train/Test Split

It's common to use anywhere between 80%-90% of your data for training/validation and save the rest for test (depending on how much data you have).

If you want to use Cross Validation or K-Fold Cross Validation, separate your validation data from your training later when you're selecting a model or tuning your model.

Everything you do from here onwards can be evaluated and tuned. Tuning is not exclusive to model hyperparameters.

VI. Prepare for ML

Prepare for ML

Now that you've split your data into training and testing, we need to turn our data into an ML-friendly format. This section is divided into 4 parts, and by the end of it, you're ready for Machine Learning.

Prepare for ML - A. Deal with Missing Data

Now we execute our imputation strategy and make sure there are no more missing values in your training and testing data.

Tip: Use scikit-learn's `SimpleImputer()` or other estimators to fill your missing values. Using estimators/transformers like these allow you to fit them to your training data then use them to transform your training and testing data.

Prepare for ML - A. Deal with Missing Data

Here's an example of filling a set of features with zero...

```
# For each feature in this array
for col in zero_fill_cols:
    # instantiate the imputer
    imputer = SimpleImputer(strategy='constant',
fill_value=0)
    # fit it to TRAINING ONLY
    imputer.fit(X_train[[col]])
    # transform BOTH training and testing
    X_train[col] = imputer.transform(X_train[[col]])
    X_test[col] = imputer.transform(X_test[[col]])
```

VII. Feature Engineering

Feature Engineering

Once you've taken care of all of your missing values, use this section as an opportunity to engineer any more features that you think will be useful for your ML model.

Again, treat this as a hyperparameter you may decide to tune.

Feature Engineering can sometimes improve your models much more than extensive model hyperparameter tuning.

Feature Engineering - Example

```
# Creating ratio of listing capacity to beds
X_train['capacity_to_beds'] =
X_train['accommodates']/X_train['beds']
X_test['capacity_to_beds'] =
X_test['accommodates']/X_test['beds']

# Careful - if you divide by zero, you create
infinite values
X_train['capacity_to_beds'] =
np.where(np.isinf(X_train['capacity_to_beds']),
            0,
            X_train['capacity_to_beds'])

X_test['capacity_to_beds'] =
np.where(np.isinf(X_test['capacity_to_beds']),
            0,
```

Feature Engineering

Note that whenever you engineer your own features (specifically, if you perform division or logarithmic operations), you open the possibility of creating invalid infinite values or NA values.

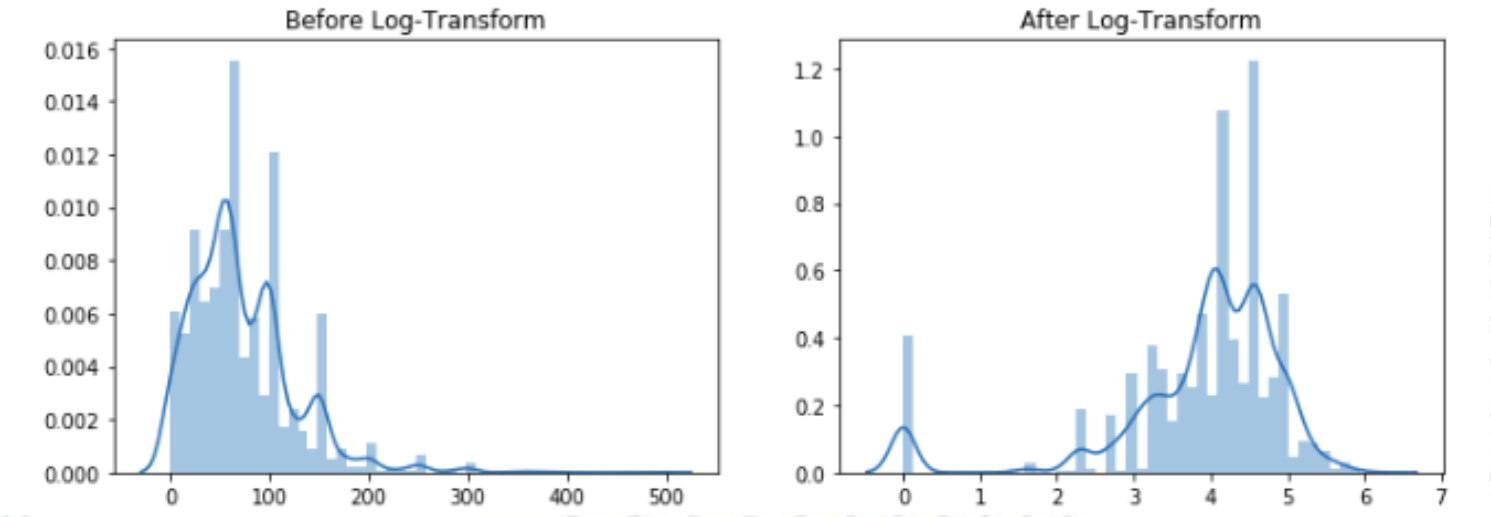
VIII. Transform Data

Transform Data

Once you're happy with the features that you have, you now have to resize/scale/transform your numerical variables or convert all of your categorical or text features into numbers.

Numerical Data

If you have very skewed data, it may be a good idea to log-transform or square-transform to make your data appear more normally distributed.



Log-Transform on cleaning_fee.

Numerical Data

If you suspect outliers negatively affect your models' performance, remove outliers now.

```
# One-sided outlier cutoff
cont_var_mean = np.mean(eda['cont_var'])
cont_var_std = np.std(eda['cont_var'])
cont_var_cutoff = cont_var_mean + cont_var_std*3

X_train = X_train[X_train['price'] < cont_var_cutoff]
```

Numerical Data

Not all ML algorithms require features to be on the same scale, but some do (Linear Regression, Neural Networks, K-Means Clustering, etc.). If this is the case for you, you have to scale your features. Some options include Standardization, Normalization, and Minimum-Maximum scaling.

```
scaler = StandardScaler()
# fitting
scaler.fit(X_train[cols_to_standardize])

# transforming
X_train[cols_to_standardize] =
scaler.transform(X_train[cols_to_standardize])
X_test[cols_to_standardize] =
scaler.transform(X_test[cols_to_standardize])
```

Categorical Data

There are generally three types of categorical data, and here is how you can approach each one:

1. Nominal (e.g. Red, Green, Blue): No inherent order to the categories. One hot encode these features using a transformer like sklearn's MultiLabelBinarizer (if values are compacted into one feature) or OneHotEncoder (if there is only one category per row for a particular feature).
2. Ordinal (e.g. Low, Med, High): Try to preserve the natural order in the data by encoding these features as ordinal numbers (i.e. 1, 2, 3). You can create a custom transformer yourself to map these values to a desired number or use sklearn's OrdinalEncoder.
3. Binary: If you haven't dealt with your binary features yet, encode them to 0 or 1 using a sklearn transformers such as `LabelBinarizer` or `MultiLabelBinarizer`.

Text Data

Without going too deep into the world of Natural Language Processing, there are generally these basic steps you can take when dealing with text data:

1. Tokenize the text using nltk's `word_tokenize` or `sentence_tokenize`. Tokenizing text data is simply breaking up words, emojis, etc into their own individual strings/objects rather than one large string.
2. Remove stop words using nltk's stopwords corpus.
3. Stem or Lemmatize your tokenized words. Stemming is trimming off letters that don't add much meaning to the word (e.g. „-ing", „-ed", etc.). Lemmatizing accomplishes stemming, but it's more computationally expensive.
4. Convert the tokens into a matrix containing either the counts of the tokens or the TF-IDF value of the tokens.

Feature Selection

Finally, now is when we perform some dimensionality reduction.

Not only can this improve your models' prediction performance, but it can also decrease computation time.

Feature Selection - Ideas

- Variance Thresholding: Omit features with low variance, since features with high variance often contain the most information for your models.
- Remove highly correlated features: Identify paired features with high correlation, and remove one of them.
- Categorical feature irrelevance? Use Chi-Square testing to remove features that are irrelevant/independent from your target feature.
- Numerical feature irrelevance? Use ANOVA F-statistic to determine features that may be irrelevant from your target feature.
- Recursive Elimination of Features

Feature Selection - Example

```
threshold = 0.8

# Calculating an absolute value correlation matrix
corr_mat = X_train.corr().abs()

# Getting upper triangle of this matrix only
upper = pd.DataFrame(np.triu(corr_mat, k=1),
columns=X_train.columns)

# Select columns with correlations above threshold
corr_col_drop = [col for col in upper.columns if
any(upper[col] > threshold)]

print(f'There are len(corr_col_drop) columns to
remove out of len(X_train.columns).')
```

Feature Selection - Example

Once you're finished with Feature Selection, we are officially ready for Machine Learning!

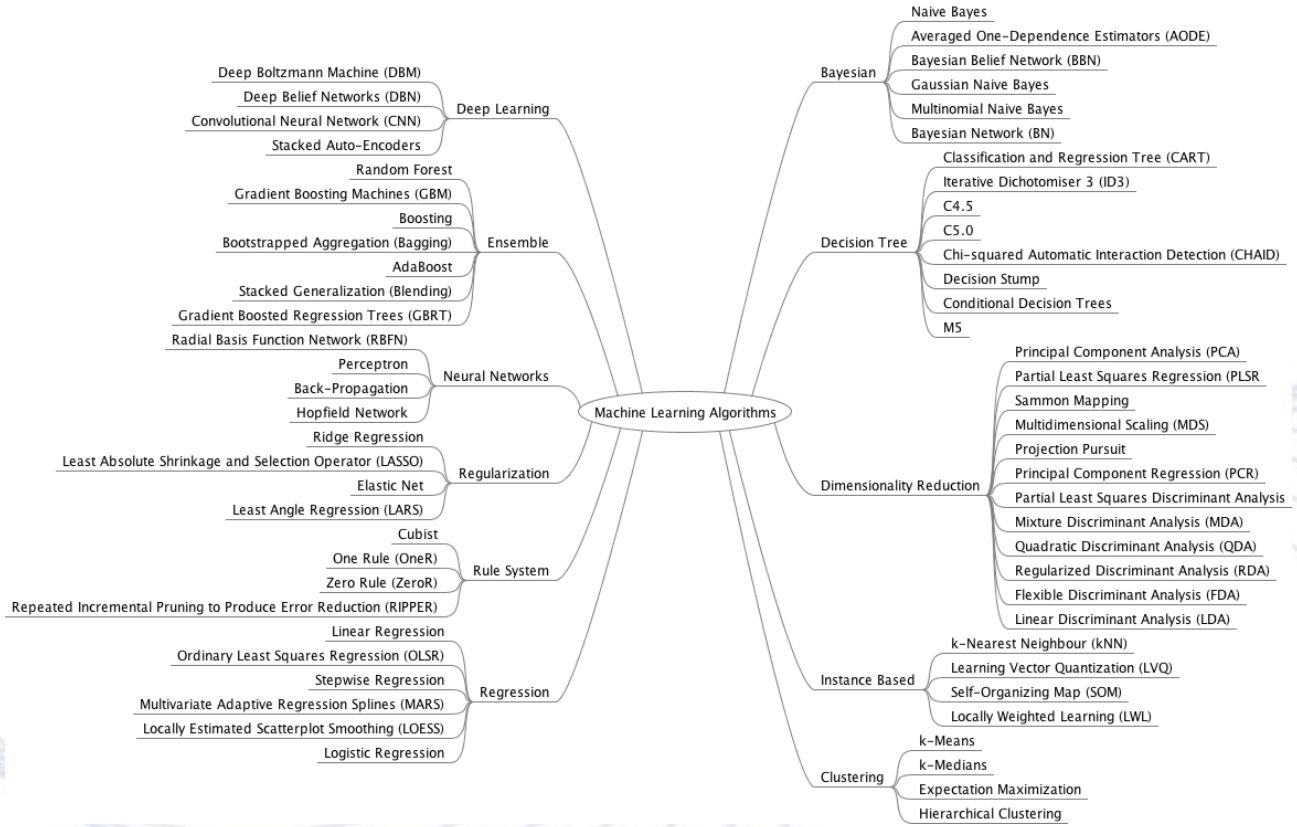
Feature Selection - Example

The next sections are brief because they describe only the fundamental steps that are taken once data is sufficiently prepped.

IX. Pick your Models

Pick your Models

Depending on your problem, pick the appropriate models that you think can be effective.



X. Model Selection

Model Selection

Once you've picked some model candidates, use methods like K-Fold Cross-validation to identify the 2-3 most promising models!

XI. Model Tuning

Model Tuning

Use standard techniques like Grid Search or Randomized Search to identify hyperparameters that perform the best.

Model Tuning

If you're up for it, explore libraries like `hyperopt` that take advantage of Bayesian statistics to tune your models. This is basically a „smart“ Randomized Search, which is basically a „faster“ Grid Search.

XII. Pick the Best Model

Pick the Best Model

Deploy, and you're done!

Concluding Thoughts

Concluding Thoughts

Notice how this blog post is 90% data cleaning, exploration, and preprocessing, but it's only 10% machine learning.

Concluding Thoughts

When you start a Machine Learning project, it can feel like it takes forever to reach a simple working model. It's fun to try different models, learn about advanced hyperparameter tuning, etc., but it can feel soul-draining to grind through datasets before you can get to that point.

Concluding Thoughts

Cleaning and preprocessing is not romanticized as the sexiest skill of the decade. However, I hope this has shown you how crucial and complex it is to stay organized and to take your time whenever you have to do this.

Use the Checklist

Checklist